UVM FIFO

2026

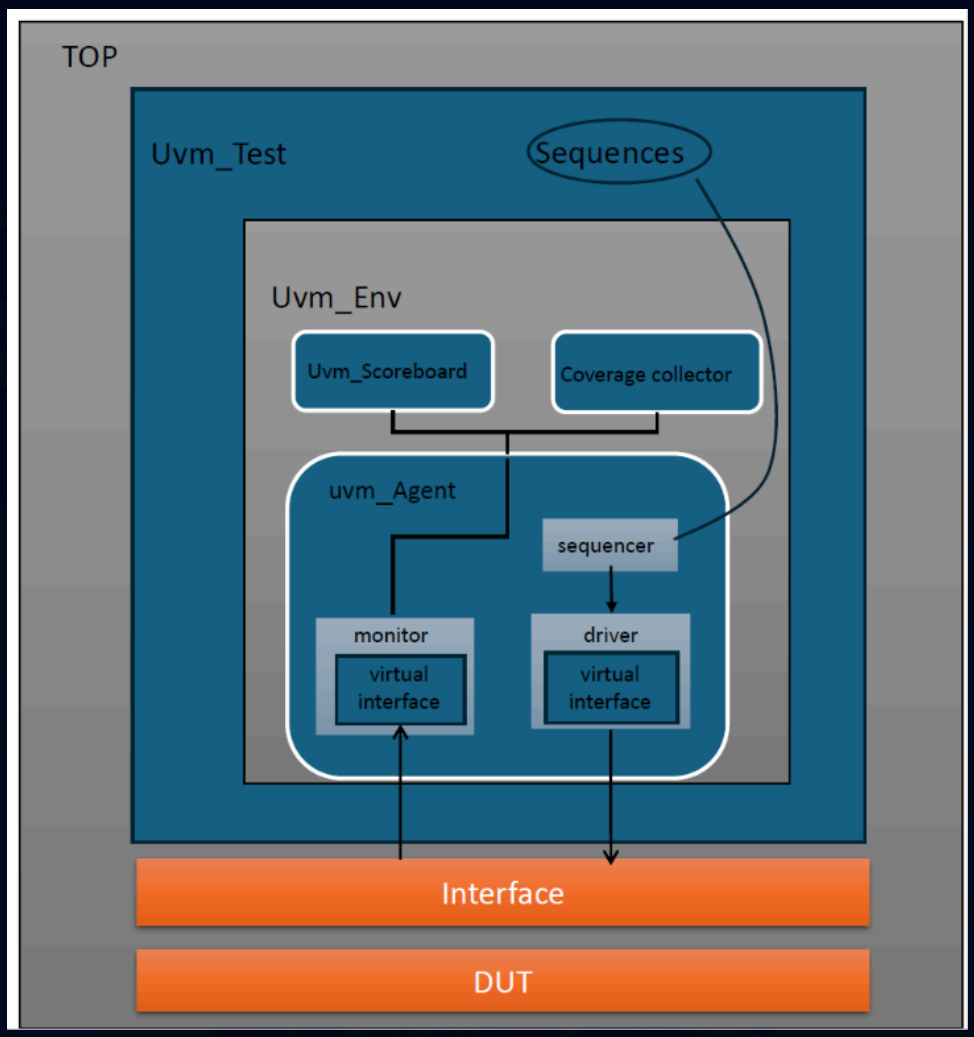
Prepared by :

Abdelrahman Mohamed Eid

Prepared for :

ENG : Kareem Waseem

OVERVIEW



UVM Structure

OVERVIEW

UVM Architecture Description:

- The top module instantiates the DUT (Design Under Test) and the physical Interface. Additionally, the top module uses the bind construct to connect the assertions module to the DUT to continuously monitor and verify the hardware behavior.

- UVM Test & Sequences: The top module calls run_test(), which dynamically creates the test component using the UVM factory and starts the UVM phases. Inside the test, the environment is built, and based on the verification plan, three main sequences are generated and executed on the sequencer: a write_only_sequence, a read_only_sequence, and a concurrent write_read_sequence

- Driving the Interface: The driver pulls these transactions from the sequencer and translates them into pin-level signals. It drives these signals onto the physical Interface via its virtual interface handle.

- Monitoring: The monitor continuously observes the pin-level activity on the Interface (also via a virtual interface). It converts these physical signals back into transaction-level objects.

- Analyzing the Output: The monitor broadcasts these sampled transactions through analysis ports to both the Uvm_Scoreboard and the Coverage collector. The scoreboard compares the actual output from the DUT against expected results (Golden Model) to check for correctness, while the coverage collector ensures all verification scenarios have been hit.

FIFO ASSERTION

```
`ifdef SIM
import Myseq_item::*;
module fifo_sva(fifo_if f_if);
always_comb begin

    if(!f_if.rst_n)
    reset :assert final (f_if.count == 0 && f_if.wr_ptr==0 && f_if.rd_ptr ==0 && f_if.data_out == 0 && f_if.wr_ack == 0 && f_if.overflow == 0 && f_if.underflow == 0 &&
f_if.empty == 1 && f_if.full == 0 && f_if.almostfull == 0 && f_if.almostempty == 0);

end


property data_out_stable;
  @(posedge f_if.clk) disable iff (!f_if.rst_n)
  !(f_if.rd_en && !f_if.empty) |=> $stable(f_if.data_out);
endproperty

property Write_Acknowledge;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_en && !f_if.full) |=> f_if.wr_ack;
endproperty

property Overflow_Detection;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_en && f_if.full && !f_if.rd_en ) |=> f_if.overflow;
endproperty

property Underflow_Detection;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.rd_en && f_if.empty && !f_if.wr_en) |=> f_if.underflow;
endproperty

property Empty_Flag;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.count == 0) |-> f_if.empty;
endproperty

property Full_Flag;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.count == f_if.FIFO_DEPTH) |-> f_if.full;
endproperty

property Almost_Full;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.count == f_if.FIFO_DEPTH-1) |-> f_if.almostfull;
endproperty

property Almost_Empty;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.count == 1) |-> f_if.almostempty;
endproperty

property Wraparound_write;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) ((f_if.wr_ptr == f_if.FIFO_DEPTH-1) && f_if.wr_en && !f_if.full) |=> !f_if.wr_ptr;
endproperty

property Wraparound_read;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) ((f_if.rd_ptr == f_if.FIFO_DEPTH-1) && f_if.rd_en && !f_if.empty) |=> !f_if.rd_ptr;
endproperty

property Pointer_threshold;
    @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.rd_ptr < f_if.FIFO_DEPTH && f_if.wr_ptr < f_if.FIFO_DEPTH && f_if.count <= f_if.FIFO_DEPTH);
endproperty
assert property (Write_Acknowledge);
cover property (Write_Acknowledge);

assert property (data_out_stable);
cover property (data_out_stable);

assert property (Overflow_Detection);
cover property (Overflow_Detection);

assert property (Underflow_Detection);
cover property (Underflow_Detection);

assert property (Empty_Flag);
cover property (Empty_Flag);

assert property (Full_Flag);
cover property (Full_Flag);

assert property (Almost_Full);
cover property (Almost_Full);

assert property (Almost_Empty);
cover property (Almost_Empty);

assert property (Wraparound_write);
cover property (Wraparound_write);

assert property (Wraparound_read)
cover property (Wraparound_read);

assert property (Pointer_threshold);
cover property (Pointer_threshold);
endmodule
`endif
```

fifo_Assertions

FIFO DESIGN

```
////////////////////////////////////////////////////////////////////////////////
// Author: Kareem Waseem
// Course: Digital Verification using SV & UVM
// Description: FIFO Design
////////////////////////////////////////////////////////////////////////////////
module FIFO(fifo_if f_if);

localparam max_fifo_addr = $clog2(f_if.FIFO_DEPTH);

reg [f_if.FIFO_WIDTH-1:0] mem [f_if.FIFO_DEPTH-1:0];

logic [max_fifo_addr-1:0] wr_ptr, rd_ptr;
logic [max_fifo_addr:0] count;
assign f_if.count=count;
assign f_if.wr_ptr=wr_ptr;
assign f_if.rd_ptr=rd_ptr;

always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        wr_ptr <= 0;
        f_if.wr_ack <=0; // modified
        f_if.overflow <=0; // modified
    end
    else if (f_if.wr_en && count < f_if.FIFO_DEPTH) begin
        mem[wr_ptr] <= f_if.data_in;
        f_if.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
        f_if.overflow <=0; // modified


    end
    else begin
        f_if.wr_ack <= 0;
        if (f_if.full & f_if.wr_en )
            f_if.overflow <= 1;
        else
            f_if.overflow <= 0;
    end
end

always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        rd_ptr <= 0;
        f_if.underflow<=0; //modified
        f_if.data_out<=0;  //modified
    end
    else if (f_if.rd_en && count != 0) begin
        f_if.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
        f_if.underflow<=0; //modified

    end
    else if(f_if.rd_en && count == 0)
        f_if.underflow<=1;

    else
            f_if.underflow<=0; //modified
end

always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        count <= 0;
    end
    else begin
        if  ( ({f_if.wr_en, f_if.rd_en} == 2'b10) && !f_if.full)
            count <= count + 1;
        else if ( ({f_if.wr_en, f_if.rd_en} == 2'b01) && !f_if.empty)
            count <= count - 1;
        else if ( ({f_if.wr_en, f_if.rd_en} == 2'b11) && f_if.empty) //added
            count <= count + 1;

        else if ( ({f_if.wr_en, f_if.rd_en} == 2'b11) && f_if.full) // added
            count <= count - 1;

    end
end

assign f_if.full = (count == f_if.FIFO_DEPTH)? 1 : 0;
assign f_if.empty = (count == 0)? 1 : 0;
assign f_if.almostfull = (count == f_if.FIFO_DEPTH-1)? 1 : 0; // modified
assign f_if.almostempty = (count == 1)? 1 : 0;

endmodule
```

FIFO DESIGN

Bugs in design

- Underflow is sequntial not combinational
- almostfull is high when count=FIFO_DEPTH-1 , not FIFO_WIDTH -2
- data_out= 0 when reset

FIFO INTERFACE &TOP

```
1
2   interface fifo_if(clk);
3   parameter FIFO_WIDTH = 16;
4   parameter FIFO_DEPTH = 8;
5   localparam max_fifo_addr = $clog2(FIFO_DEPTH);
6
7   input clk;
8   logic [FIFO_WIDTH-1:0] data_in;
9   logic  rst_n, wr_en, rd_en;
10  logic  [FIFO_WIDTH-1:0] data_out;
11  logic  wr_ack, overflow;
12  logic full, empty, almostfull, almostempty, underflow;
13  logic [max_fifo_addr-1:0] wr_ptr, rd_ptr;
14  logic [max_fifo_addr:0] count;
15
16  endinterface //interfacename
```

Interface

top.sv

```
1
2   import uvm_pkg::*;
3   `include "uvm_macros.svh"
4   import fifo_test::*;
5   module top();
6
7     bit clk;
8     always #2 clk = ~clk;
9
10    fifo_if f (clk);
11
12
13  FIFO DUT (f);
14
15  bind FIFO fifo_sva fifo_assertions (f_if);
16
17     initial begin
18      uvm_config_db#(virtual fifo_if)::set(null,"uvm_test_top","fifo_if",f);
19    run_test("fifo_test");
20    end
21
22
23  endmodule
```

TOP

FIFO TEST

```
package fifo_test;
import uvm_pkg::*;
`include "uvm_macros.svh"

import fifo_env::*;
import fifo_config::*;
import write_only_seq::*;
import write_read_seq::*;
import read_only_seq::*;
import rst_seq::*;



class fifo_test extends uvm_test;
  `uvm_component_utils(fifo_test)

  function new(string name,uvm_component parent);
    super.new (name,parent);
  endfunction

  fifo_env tst_env;
  fifo_config tst_config;
  write_only_seq write_seq;
  read_only_seq read_seq;
  write_read_seq w_r_seq;
  rst_seq r_seq;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    tst_env = fifo_env::type_id::create("tst_env",this);
     write_seq   = write_only_seq::type_id::create("write_seq",this);
     read_seq    = read_only_seq::type_id::create("read_seq",this);
     w_r_seq    = write_read_seq::type_id::create("write_read_seq",this);
     r_seq    = rst_seq::type_id::create("r_seq",this);



    tst_config = fifo_config::type_id::create("fifo_config");

    if(!uvm_config_db#(virtual
fifo_if)::get(this,"","vif",tst_config.vif))`uvm_fatal("build_phase","Test config get the virtual interface");

    uvm_config_db#(fifo_config)::set(this,"*","SCF",tst_config);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);

    phase.raise_objection(this);

    `uvm_info("run phase","Reset asserted ",UVM_LOW);
    r_seq.start(tst_env.env_agt.agt_sqr);
    `uvm_info("run phase","Reset Deasserted ",UVM_LOW);

    `uvm_info("run phase","Write sequence Started ",UVM_LOW);
    write_seq.start(tst_env.env_agt.agt_sqr);
    `uvm_info("run phase","Write sequence Finished ",UVM_LOW);

    `uvm_info("run phase","read sequence Started ",UVM_LOW);
    read_seq.start(tst_env.env_agt.agt_sqr);
    `uvm_info("run phase","read sequence Finished ",UVM_LOW);

    `uvm_info("run phase","Write&read sequence Started ",UVM_LOW);
    w_r_seq.start(tst_env.env_agt.agt_sqr);
    `uvm_info("run phase","Write&read sequence Finished ",UVM_LOW);

    phase.drop_objection(this);
  endtask

endclass: fifo_test
endpackage
```

FIFO ENV

```
package fifo_env;
import uvm_pkg::*;
`include "uvm_macros.svh"
import fifo_agent::*;
import fifo_sb::*;
import fifo_coverage::*;

class fifo_env extends uvm_env;
  `uvm_component_utils(fifo_env)

  fifo_scoreboard env_sb;
  fifo_agent env_agt;
  fifo_coverage env_cvg;

  function new(string name = "fifo_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env_sb=fifo_scoreboard::type_id::create("score_board",this);
    env_agt=fifo_agent::type_id::create("agent",this);
    env_cvg=fifo_coverage::type_id::create("env_cvg",this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    env_agt.agt_ap.connect(env_sb.sb_ae);
    env_agt.agt_ap.connect(env_cvg.cvg_ae);
  endfunction
endclass
endpackage
```

fifo_config.sv

```
package fifo_config;
import uvm_pkg::*;
`include "uvm_macros.svh"
class fifo_config extends uvm_object;
  `uvm_object_utils(fifo_config)

  virtual fifo_if vif;
  function new(string name="fifo_config");
    super.new(name);
  endfunction
endclass
endpackage

```

fifo_config

FIFO AGENT

```
package fifo_agent;
import uvm_pkg::*;
`include "uvm_macros.svh"
import fifo_driver::*;
import fifo_monitor::*;
import fifo_sequencer::*;
import Myseq_item::*;
import fifo_config::*;

class fifo_agent extends uvm_agent;
  `uvm_component_utils(fifo_agent)
  fifo_sequencer agt_sqr;
  fifo_driver agt_drv;
  fifo_monitor agt_mon;
  fifo_config agt_cfg;
  uvm_analysis_port#(Myseq_item) agt_ap;

  function new(string name="agent",uvm_component parent);
      super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase );
    super.build_phase(phase);
    agt_sqr=fifo_sequencer::type_id::create("agt_sqr",this);
    agt_drv=fifo_driver::type_id::create("agt_drv",this);
    agt_mon=fifo_monitor::type_id::create("agt_mon",this);
    agt_ap=new("agt_ap",this);

    if(!uvm_config_db#(fifo_config):: get(this,"","SCF",agt_cfg))
          `uvm_fatal("build_phase"," unable to get the config object");
  endfunction

  function void connect_phase(uvm_phase phase);
      super.connect_phase(phase);
      agt_drv.seq_item_port.connect(agt_sqr.seq_item_export);
      agt_drv.vif=agt_cfg.vif;
      agt_mon.vif=agt_cfg.vif;
      agt_mon.mon_ap.connect(agt_ap);
  endfunction
endclass
endpackage
```

FIFO MONITOR

```
package fifo_monitor;
import uvm_pkg::*;
`include "uvm_macros.svh"
import Myseq_item::*;

class fifo_monitor extends uvm_monitor;
  `uvm_component_utils(fifo_monitor)

  virtual fifo_if vif;
  Myseq_item rsp_seq_item;
  uvm_analysis_port#(Myseq_item) mon_ap;

  function new(string name="monitor",uvm_component parent);
      super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
      super.build_phase(phase);
      mon_ap=new("mon_ap",this);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      rsp_seq_item =
Myseq_item::type_id::create("rsp_seq_item");
      @(negedge vif.clk);

      rsp_seq_item.rst_n   = vif.rst_n;
      rsp_seq_item.wr_en   = vif.wr_en;
      rsp_seq_item.rd_en   = vif.rd_en;
      rsp_seq_item.data_in = vif.data_in;

      rsp_seq_item.data_out    = vif.data_out;
      rsp_seq_item.wr_ack      = vif.wr_ack;
      rsp_seq_item.overflow    = vif.overflow;
      rsp_seq_item.full        = vif.full;
      rsp_seq_item.empty       = vif.empty;
      rsp_seq_item.almostfull  = vif.almostfull;
      rsp_seq_item.almostempty = vif.almostempty;
      rsp_seq_item.underflow   = vif.underflow;

      mon_ap.write(rsp_seq_item);
    end
  endtask
endclass
endpackage
```

FIFO DRIVER

```
1  package fifo_driver;
2  import uvm_pkg::*;
3  `include "uvm_macros.svh"
4  import Myseq_item::*;
5
6  class fifo_driver extends uvm_driver #(Myseq_item);
7      `uvm_component_utils(fifo_driver)
8      virtual fifo_if vif;
9      Myseq_item seq_item;
10
11     function new(string name="fifo_driver", uvm_component parent);
12       super.new(name,parent);
13     endfunction
14
15     task run_phase(uvm_phase phase);
16         super.run_phase(phase);
17
18         forever begin
19           seq_item_port.get_next_item(seq_item);
20
21           vif.rst_n   = seq_item.rst_n;
22           vif.wr_en   = seq_item.wr_en;
23           vif.rd_en   = seq_item.rd_en;
24           vif.data_in = seq_item.data_in;
25
26           @(negedge vif.clk);
27           seq_item_port.item_done();
28         end
29     endtask
30
31 endclass
32 endpackage
```

FIFO SQR & SEQ_ITEM

fifo_sqr.sv

```
1   package fifo_sequencer;
2   import uvm_pkg::*;
3   `include "uvm_macros.svh"
4   import Myseq_item::*;
5
6   class fifo_sequencer extends uvm_sequencer#(Myseq_item);
7     `uvm_component_utils(fifo_sequencer)
8
9     function new(string name="sqr",uvm_component parent);
10        super.new(name,parent);
11    endfunction
12  endclass
13  endpackage
```

fifo_sqr

fifo_seq_item.sv

```
1   package Myseq_item;
2   import uvm_pkg::*;
3   `include "uvm_macros.svh";
4
5   parameter FIFO_WIDTH = 16;
6   parameter FIFO_DEPTH = 8;
7   class Myseq_item extends uvm_sequence_item;
8   `uvm_object_utils(Myseq_item);
9
10  rand logic [FIFO_WIDTH-1:0] data_in;
11  rand logic  rst_n, wr_en, rd_en;
12   logic  [FIFO_WIDTH-1:0] data_out;
13   logic  wr_ack, overflow;
14   logic full, empty, almostfull, almostempty, underflow;
15
16
17    function new(string name="seq_item");
18      super.new(name);
19    endfunction
20
21  constraint c1{
22      rst_n dist {1:=98 , 0:=2};
23  }
24
25  function string convert2string();
26    return $sformatf("rst_n=%0b wr_en=%0b rd_en=%0b data_in=0x%0h | data_out=0x%0h wr_ack=%0b overflow=%0b full=%0b empty=%0b almost
27                      rst_n, wr_en, rd_en, data_in, data_out, wr_ack, overflow, full, empty, almostfull, almostempty, underflow);
28  endfunction
29  endclass
30  endpackage
```

fifo_seq_item

FIFO SEQUENCES

write_only_seq.sv

```
1
2   package write_only_seq;
3    import uvm_pkg::*;
4   `include "uvm_macros.svh"
5    import Myseq_item::*;
6   class write_only_seq extends uvm_sequence#(Myseq_item);
7
8   `uvm_object_utils(write_only_seq);
9   function new(string name="write only seq");
10      super.new(name);
11  endfunction
12  Myseq_item seq_item;
13
14  task body();
15      seq_item=Myseq_item::type_id::create("seq_item");
16      repeat(20)begin
17          start_item(seq_item);
18          assert(seq_item.randomize() with
19          {
20              seq_item.wr_en==1;
21              seq_item.rd_en==0;
22
23
24           });
25          finish_item(seq_item);
26      end
27  endtask
28  endclass
29  endpackage
```

Write_seq

read_only_seq.sv

```
1
2   package read_only_seq;
3    import uvm_pkg::*;
4   `include "uvm_macros.svh"
5    import Myseq_item::*;
6
7   class read_only_seq extends uvm_sequence#(Myseq_item);
8
9   `uvm_object_utils(read_only_seq);
10  function new(string name="read only seq");
11      super.new(name);
12  endfunction
13  Myseq_item seq_item;
14
15  task body();
16      seq_item=Myseq_item::type_id::create("seq_item");
17      repeat(20)begin
18          start_item(seq_item);
19          assert(seq_item.randomize() with
20          {
21              seq_item.wr_en==0;
22              seq_item.rd_en==1;
23           });
24          finish_item(seq_item);
25      end
26  endtask
27  endclass
28  endpackage
```

read_seq

FIFO SEQUENCES

write_read_seq.sv

```
1   package write_read_seq;
2    import uvm_pkg::*;
3   `include "uvm_macros.svh"
4    import Myseq_item::*;
5   class write_read_seq extends uvm_sequence#(Myseq_item);
6
7   `uvm_object_utils(write_read_seq);
8   function new(string name="write_read_seq");
9       super.new(name);
10  endfunction
11  Myseq_item seq_item;
12
13  task body();
14      seq_item=Myseq_item::type_id::create("seq_item");
15      repeat(500)begin
16          start_item(seq_item);
17          assert(seq_item.randomize() with
18          {
19              seq_item.wr_en dist{1:=70 , 0:=30};
20              seq_item.rd_en  dist{1:=50 , 0:=50 };
21
22
23          });
24          finish_item(seq_item);
25      end
26  endtask
27  endclass
28  endpackage
```

Write_read_seq

FIFO SCOREBOARD

```
package fifo_sb;
import uvm_pkg::*;
`include "uvm_macros.svh";
import Myseq_item::*;

class fifo_scoreboard extends uvm_scoreboard;
`uvm_component_utils(fifo_scoreboard);

uvm_analysis_export#(Myseq_item) sb_ae;
Myseq_item seq_item;
uvm_tlm_analysis_fifo#(Myseq_item)sb_fifo;
int correct_count=0;
int error_count=0;

reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
int count=0;
int wr_ptr=0,rd_ptr=0;
logic   empty_ref, full_ref;
logic  [FIFO_WIDTH-1:0] data_out_ref;


function new(string name="scoreboard",uvm_component parent);
    super.new(name,parent);
endfunction


function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    sb_fifo=new("fifo",this);
    sb_ae=new("sb_ae",this);
endfunction


function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    sb_ae.connect(sb_fifo.analysis_export);
endfunction


task run_phase(uvm_phase phase);
    super.run_phase(phase);


forever begin
    sb_fifo.get(seq_item);

    ref_model(seq_item);

    if(seq_item.data_out !== data_out_ref ) begin
        `uvm_error("run_phase",$sformatf("comparison failed , Transaction recieved by the DUT :%s while the refrence out =%0b",seq_item.convert2string(),data_out_ref));
        error_count++;
    end
    else begin
    `uvm_info("run_phase",$sformatf("correct out : %s",seq_item.convert2string()),UVM_HIGH);
    correct_count++;
    end
end
endtask

task ref_model(Myseq_item seq_item);
  if(!seq_item.rst_n) begin
            empty_ref=1;
            full_ref=0 ;
            data_out_ref=0;
            count=0;
            wr_ptr=0;
            rd_ptr=0;
        end
        else begin
            //empty
            if(count==0)
            empty_ref=1;
            else
            empty_ref=0;
            //full
            if(count==FIFO_DEPTH)
                full_ref=1;
            else
                full_ref=0;

            //write
            if(!full_ref && seq_item.wr_en ) begin
            mem [wr_ptr] =seq_item.data_in;
            count++;
            wr_ptr= (wr_ptr+1) % (FIFO_DEPTH);
            end
            //read
             if(!empty_ref && seq_item.rd_en) begin
                count--;
                data_out_ref=mem[rd_ptr];
                rd_ptr= (rd_ptr+1) % (FIFO_DEPTH);

            end

        end
    endtask

function void report_phase(uvm_phase phase);
     super.report_phase(phase);
     `uvm_info("report phase",$sformatf("Total Successful Transactions :%0d",correct_count),UVM_MEDIUM);
     `uvm_info("report phase",$sformatf("Total Failed Transactions :%0d",error_count),UVM_MEDIUM);
endfunction

endclass
endpackage
```

FIFO COVERAGE

```
package fifo_coverage;
import uvm_pkg::*;
`include "uvm_macros.svh";
import Myseq_item::*;

class fifo_coverage extends uvm_component;
`uvm_component_utils(fifo_coverage);
uvm_analysis_export#(Myseq_item) cvg_ae;
uvm_tlm_analysis_fifo#(Myseq_item) cvg_fifo;
Myseq_item cvg_seq_item;

covergroup cvr_gp;

    cp_wr_en     : coverpoint cvg_seq_item.wr_en;
    cp_rd_en     : coverpoint cvg_seq_item.rd_en;
    cp_full      : coverpoint cvg_seq_item.full;
    cp_almostfull: coverpoint cvg_seq_item.almostfull;
    cp_empty     : coverpoint cvg_seq_item.empty;
    cp_almostempty: coverpoint cvg_seq_item.almostempty;
    cp_overflow  : coverpoint cvg_seq_item.overflow;
    cp_underflow : coverpoint cvg_seq_item.underflow;
    cp_wr_ack    : coverpoint cvg_seq_item.wr_ack;

    full: cross cp_wr_en, cp_rd_en, cp_full {
        ignore_bins ign_full = binsof(cp_full) intersect {1} && binsof(cp_rd_en) intersect {1};
    }

    almostfull: cross cp_wr_en, cp_rd_en, cp_almostfull;



    almostempty: cross cp_wr_en, cp_rd_en, cp_almostempty;

    overflow: cross cp_wr_en, cp_rd_en, cp_overflow {
        ignore_bins ign_ovf1 = binsof(cp_overflow) intersect {1} && binsof(cp_rd_en) intersect {1};
        ignore_bins ign_ovf2 = binsof(cp_overflow) intersect {1} && binsof(cp_wr_en) intersect {0};
    }

    underflow: cross cp_wr_en, cp_rd_en, cp_underflow {
        ignore_bins ign_unf1 = binsof(cp_underflow) intersect {1} && binsof(cp_rd_en) intersect {0};
        ignore_bins ign_unf2 = binsof(cp_underflow) intersect {1} && binsof(cp_wr_en) intersect {1};
    }

    wr_ack: cross cp_wr_en, cp_rd_en, cp_wr_ack {
        ignore_bins ign_ack = binsof(cp_wr_ack) intersect {1} && binsof(cp_wr_en) intersect {0};
    }

endgroup

function new(string name="fifo coverage",uvm_component parent);
    super.new(name,parent);
    cvr_gp=new();
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);
cvg_ae=new("cvg_ae",this);
cvg_fifo=new("cvg_fifo",this);

endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
cvg_ae.connect(cvg_fifo.analysis_export);

endfunction
task run_phase(uvm_phase phase);
forever begin
cvg_fifo.get(cvg_seq_item);
cvr_gp.sample();
end
endtask


endclass

endpackage
```

DO FILE && COVERAGE

run.do

```
vlib work

vlog FIFO.sv fifo_if.sv +cover=sbt -covercells
vlog -f src_files.f +define+SIM
vsim -voptargs=+acc work.top -cover

add wave -position insertpoint sim:/top/f/*

run -all

coverage save fifo.ucdb -du FIFO -du fifo_sva -du fifo_coverage -du fifo_if
vcover report fifo.ucdb -details -annotate -all -output coverage_rpt_fifo.txt
```

DO FILE

```
Coverage Report by instance with details

=================================================================================
=== Instance: /\top#f
=== Design Unit: work.fifo_if
=================================================================================
Toggle Coverage:
    Enabled Coverage             Bins      Hits    Misses  Coverage
    ----------------             ----      ----    ------  --------
    Toggles                       106       106         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\top#f  --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                   almostempty           1           1      100.00
                                    almostfull           1           1      100.00
                                           clk           1           1      100.00
                                      count[3-0]         1           1      100.00
                                    data_in[15-0]        1           1      100.00
                                   data_out[15-0]        1           1      100.00
                                         empty           1           1      100.00
                                          full           1           1      100.00
                                      overflow           1           1      100.00
                                         rd_en           1           1      100.00
                                    rd_ptr[2-0]          1           1      100.00
                                         rst_n           1           1      100.00
                                     underflow           1           1      100.00
                                        wr_ack           1           1      100.00
                                         wr_en           1           1      100.00
                                    wr_ptr[2-0]          1           1      100.00

Total Node Count     =         53
Toggled Node Count   =         53
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (106 of 106 bins)
```

Total Coverage

Coverage Report in folder

COVERAGE

```
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        24        24         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\top#DUT

    Line         Item                     Count     Source
    ----         ----                     -----     ------
  File FIFO.sv
------------------------------------IF Branch------------------------------------
    19                                      551     Count coming in to IF
    19             1                         20       if (!f_if.rst_n) begin
    24             1                        297       else if (f_if.wr_en && count < f_if.FIFO_DEPTH) begin
    32             1                        234       else begin
Branch totals: 3 hits of 3 branches = 100.00%

------------------------------------IF Branch------------------------------------
    34                                      234     Count coming in to IF
    34             1                         79               if (f_if.full & f_if.wr_en )
    36             1                        155               else
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    42                                      551     Count coming in to IF
    42             1                         20       if (!f_if.rst_n) begin
    47             1                        248       else if (f_if.rd_en && count != 0) begin
    53             1                         28       else if(f_if.rd_en && count == 0)
    56             1                        255       else
Branch totals: 4 hits of 4 branches = 100.00%

------------------------------------IF Branch------------------------------------
    61                                      468     Count coming in to IF
    61             1                         20       if (!f_if.rst_n) begin
    64             1                        448       else begin
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    65                                      448     Count coming in to IF
    65             1                        153               if      ( ((f_if.wr_en, f_if.rd_en) == 2'b10) && !f_if.full)
    67             1                         79               else if ( ((f_if.wr_en, f_if.rd_en) == 2'b01) && !f_if.empty)
    69             1                         12               else if ( ((f_if.wr_en, f_if.rd_en) == 2'b11) && f_if.empty) //added
    72             1                         37               else if ( ((f_if.wr_en, f_if.rd_en) == 2'b11) && f_if.full) // added
                                            167     All False Count
Branch totals: 5 hits of 5 branches = 100.00%

------------------------------------IF Branch------------------------------------
    78                                      291     Count coming in to IF
    78             1                         57     assign f_if.full = (count == f_if.FIFO_DEPTH)? 1 : 0;
    78             2                        234     assign f_if.full = (count == f_if.FIFO_DEPTH)? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    79                                      291     Count coming in to IF
    79             1                         17     assign f_if.empty = (count == 0)? 1 : 0;
    79             2                        274     assign f_if.empty = (count == 0)? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    80                                      291     Count coming in to IF
    80             1                         93     assign f_if.almostfull = (count == f_if.FIFO_DEPTH-1)? 1 : 0; // modified
    80             2                        198     assign f_if.almostfull = (count == f_if.FIFO_DEPTH-1)? 1 : 0; // modified
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    81                                      291     Count coming in to IF
    81             1                         22     assign f_if.almostempty = (count == 1)? 1 : 0;
    81             2                        269     assign f_if.almostempty = (count == 1)? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%
```

Branch

COVERAGE

```
Statement Coverage:
    Enabled Coverage             Bins      Hits    Misses  Coverage
    ----------------             ----      ----    ------  --------
    Statements                     30        30         0   100.00%

================================Statement Details================================

Statement Coverage for instance /\top#DUT  --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FIFO.sv
    6                                                module FIFO(fifo_if f_if);
    7
    8                                                localparam max_fifo_addr = $clog2(f_if.FIFO_DEPTH);
    9
    10                                               reg [f_if.FIFO_WIDTH-1:0] mem [f_if.FIFO_DEPTH-1:0];
    11
    12                                               logic [max_fifo_addr-1:0] wr_ptr, rd_ptr;
    13                                               logic [max_fifo_addr:0] count;
    14                                               assign f_if.count=count;
    15                                               assign f_if.wr_ptr=wr_ptr;
    16                                               assign f_if.rd_ptr=rd_ptr;
    17
    18              1                          551   always @(posedge f_if.clk or negedge f_if.rst_n) begin
    19                                                  if (!f_if.rst_n) begin
    20              1                           20             wr_ptr <= 0;
    21              1                           20             f_if.wr_ack <=0; // modified
    22              1                           20             f_if.overflow <=0; // modified
    23                                                  end
    24                                                  else if (f_if.wr_en && count < f_if.FIFO_DEPTH) begin
    25              1                          297             mem[wr_ptr] <= f_if.data_in;
    26              1                          297             f_if.wr_ack <= 1;
    27              1                          297             wr_ptr <= wr_ptr + 1;
    28              1                          297             f_if.overflow <=0; // modified
    29
    30
    31                                                  end
    32                                                  else begin
    33              1                          234             f_if.wr_ack <= 0;
    34                                                          if (f_if.full & f_if.wr_en )
    35              1                           79                     f_if.overflow <= 1;
    36                                                          else
    37              1                          155                     f_if.overflow <= 0;
    38                                                  end
    39                                          end
    40
    41              1                          551   always @(posedge f_if.clk or negedge f_if.rst_n) begin
    42                                                  if (!f_if.rst_n) begin
    43              1                           20             rd_ptr <= 0;
    44              1                           20             f_if.underflow<=0; //modified
    45              1                           20             f_if.data_out<=0;  //modified
    46                                                  end
    47                                                  else if (f_if.rd_en && count != 0) begin
    48              1                          248             f_if.data_out <= mem[rd_ptr];
    49              1                          248             rd_ptr <= rd_ptr + 1;
    50              1                          248             f_if.underflow<=0; //modified
    51
    52                                                  end
    53                                                  else if(f_if.rd_en && count == 0)
    54              1                           28             f_if.underflow<=1;
    55
    56                                                  else
    57              1                          255                     f_if.underflow<=0; //modified
    58                                          end
    59
    60              1                          468   always @(posedge f_if.clk or negedge f_if.rst_n) begin
    61                                                  if (!f_if.rst_n) begin
    62              1                           20             count <= 0;
    63                                                  end
    64                                                  else begin
    65                                                          if      ( ({f_if.wr_en, f_if.rd_en} == 2'b10) && !f_if.full)
    66              1                          153                     count <= count + 1;
    67                                                  else if ( ({f_if.wr_en, f_if.rd_en} == 2'b01) && !f_if.empty)
    68              1                           79                     count <= count - 1;
    69                                                  else if ( ({f_if.wr_en, f_if.rd_en} == 2'b11) && f_if.empty) //added
    70              1                           12                     count <= count + 1;
```

Statement

COVERAGE

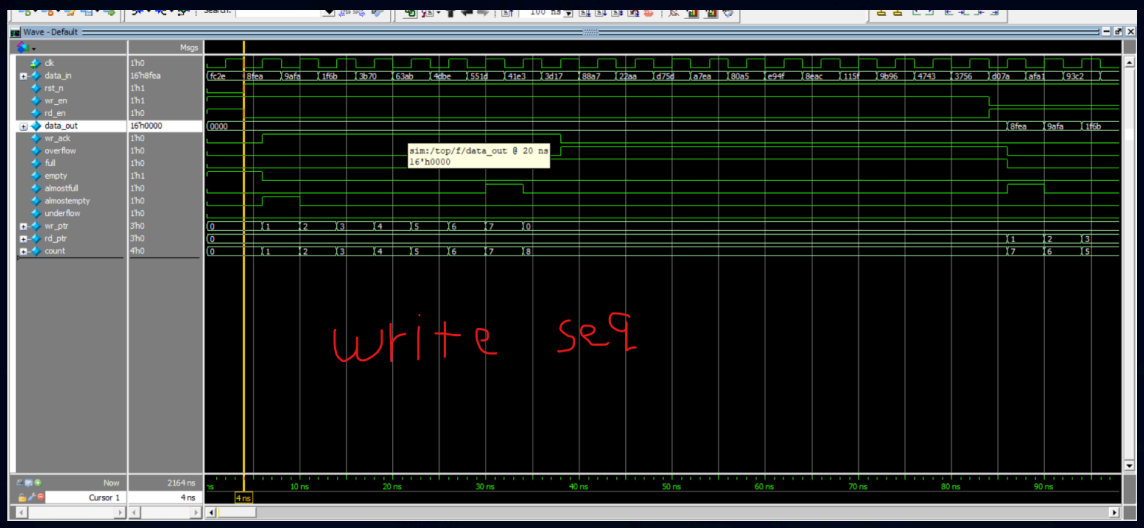
```
TOTAL DIRECTIVE COVERAGE: 100.00%  COVERS: 22

ASSERTION RESULTS:
--------------------------------------------------------------------
Name                 File(Line)                    Failure      Pass
                                                   Count        Count
--------------------------------------------------------------------
/\top#DUT /fifo_assertions/assert__Pointer_threshold
                     fifo_sva.sv(87)                 0          1
/\top#DUT /fifo_assertions/assert__Wraparound_read
                     fifo_sva.sv(84)                 0          1
/\top#DUT /fifo_assertions/assert__Wraparound_write
                     fifo_sva.sv(81)                 0          1
/\top#DUT /fifo_assertions/assert__Almost_Empty
                     fifo_sva.sv(78)                 0          1
/\top#DUT /fifo_assertions/assert__Almost_Full
                     fifo_sva.sv(75)                 0          1
/\top#DUT /fifo_assertions/assert__Full_Flag
                     fifo_sva.sv(72)                 0          1
/\top#DUT /fifo_assertions/assert__Empty_Flag
                     fifo_sva.sv(69)                 0          1
/\top#DUT /fifo_assertions/assert__Underflow_Detection
                     fifo_sva.sv(66)                 0          1
/\top#DUT /fifo_assertions/assert__Overflow_Detection
                     fifo_sva.sv(63)                 0          1
/\top#DUT /fifo_assertions/assert__data_out_stable
                     fifo_sva.sv(60)                 0          1
/\top#DUT /fifo_assertions/assert__Write_Acknowledge
                     fifo_sva.sv(57)                 0          1
/\top#DUT /fifo_assertions/reset
                     fifo_sva.sv(7)                  0          1
/\top#DUT#fifo_assertions /assert__Pointer_threshold
                     fifo_sva.sv(87)                 0          1
/\top#DUT#fifo_assertions /assert__Wraparound_read
                     fifo_sva.sv(84)                 0          1
/\top#DUT#fifo_assertions /assert__Wraparound_write
                     fifo_sva.sv(81)                 0          1
/\top#DUT#fifo_assertions /assert__Almost_Empty
                     fifo_sva.sv(78)                 0          1
/\top#DUT#fifo_assertions /assert__Almost_Full
                     fifo_sva.sv(75)                 0          1
/\top#DUT#fifo_assertions /assert__Full_Flag
                     fifo_sva.sv(72)                 0          1
/\top#DUT#fifo_assertions /assert__Empty_Flag
                     fifo_sva.sv(69)                 0          1
/\top#DUT#fifo_assertions /assert__Underflow_Detection
                     fifo_sva.sv(66)                 0          1
/\top#DUT#fifo_assertions /assert__Overflow_Detection
                     fifo_sva.sv(63)                 0          1
/\top#DUT#fifo_assertions /assert__data_out_stable
                     fifo_sva.sv(60)                 0          1
/\top#DUT#fifo_assertions /assert__Write_Acknowledge
                     fifo_sva.sv(57)                 0          1
/\top#DUT#fifo_assertions /reset
                     fifo_sva.sv(7)                  0          1

Total Coverage By Instance (filtered view): 100.00%
```

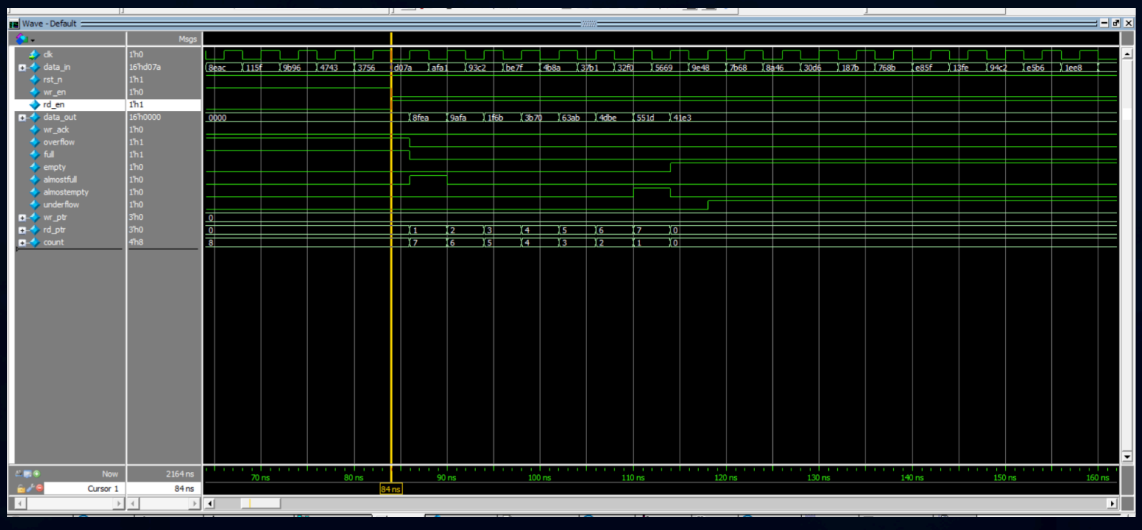
Assertion &Total coverage

WAVEFORM SNIPTTES



Write_seq

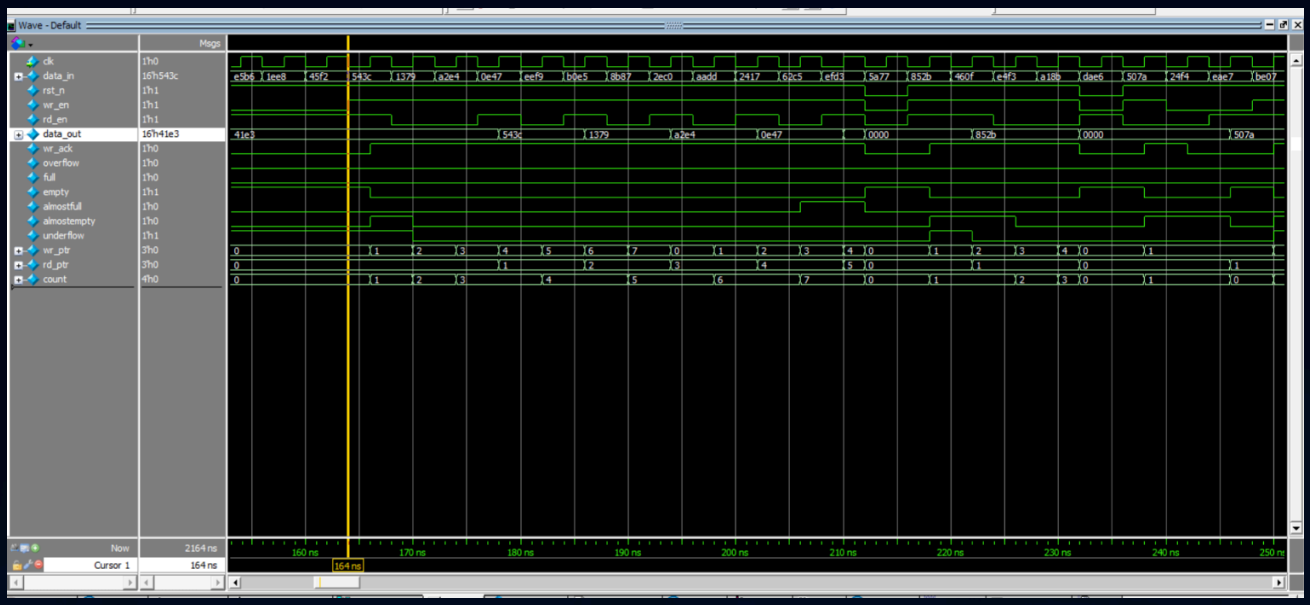
The driver asserts wr_en=1 and rd_en=0 while feeding random data to data_in. This increases wr_ptr and count, and triggers wr_ack



read_seq

The driver asserts rd_en=1 and wr_en=0 to pop data. This outputs data on data_out, increases rd_ptr, and decreases count

WAVEFORM SNIPTTES



write_read_seq

The driver asserts both wr_en and rd_en based on randomization. This tests the FIFO's behavior under simultaneous read and write operations

VF PLAN

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	When the reset is asserted, the internal counters and pointers should be low	Executed via reset_sequence. Sequence item constraints keep reset deasserted during normal operation.	N/A	Immediate assertion in interface to make sure internal counters and pointers are set to 0
FIFO_2	When the wr_en is asserted && full is low, the value on data_in should be stored in FIFO	Executed via write_only_sequence and write_read_sequence from the sequencer.	Coverage_Collector class covers all permutations of wr_en and rd_en with all outputs.	Uvm_Scoreboard compares data with Golden Model. Concurrent assertion to check wr_ack
FIFO_3	When the wr_en is asserted && full is high, overflow should be high	Executed via write_only_sequence and write_read_sequence from the sequencer.	Coverage_Collector class covers all permutations of wr_en and rd_en with all outputs.	Concurrent assertion to check overflow
FIFO_4	When the rd_en is asserted && empty is low, data_out should equal the value stored in FIFO in location rd_ptr	Executed via read_only_sequence and write_read_sequence from the sequencer.	Coverage_Collector class covers all permutations of wr_en and rd_en with all outputs.	Uvm_Scoreboard receives transaction items via analysis ports and compares data_out against Golden Model.
FIFO_5	When the rd_en is asserted && empty is high, underflow should be high	Executed via read_only_sequence and write_read_sequence from the sequencer.	Coverage_Collector class covers all permutations of wr_en and rd_en with all outputs.	Concurrent assertion to check underflow
FIFO_6	When the wr_en is asserted && rd_en is asserted && full is high, only read will take place and data_out should equal the value stored	Executed via write_read_sequence from the sequencer.	Coverage_Collector class covers all permutations of wr_en and rd_en with all outputs.	Uvm_Scoreboard compares data with Golden Model.
FIFO_7	When the wr_en is asserted && rd_en is asserted && empty is high, only write will take place and the value on data_in should be stored	Executed via write_read_sequence from the sequencer.	Coverage_Collector class covers all permutations of wr_en and rd_en with all outputs.	Concurrent assertion to check wr_ack flag
FIFO_8	When the internal count equals the FIFO depth, the full flag should be asserted	Executed via write_only_sequence and write_read_sequence.	Coverage_Collector class covers all permutations of wr_en and rd_en with all outputs.	Concurrent assertion to check full flag
FIFO_9	When the internal count equals the 0, the empty flag should be asserted	Executed via read_only_sequence and write_read_sequence.	Coverage_Collector class covers all permutations of wr_en and rd_en with all outputs.	Concurrent assertion to check empty flag
FIFO_10	When the internal count equals 1, the almostempty flag should be asserted	Executed via read_only_sequence and write_read_sequence.	Coverage_Collector class covers all permutations of wr_en and rd_en with all outputs.	Concurrent assertion to check almostempty flag

Verification Plan

Feature	Assertion
When reset (rst_n) is asserted, all outputs and pointers return to their reset values (count, wr_ptr, rd_ptr, data_out, wr_ack, overflow, underflow, full, almostfull, almostempty = 0 and empty = 1)	always_comb if (!rst_n) assert final (count == 0 && wr_ptr == 0 && rd_ptr == 0 && data_out == 0 && wr_ack == 0 && overflow == 0 && underflow == 0 && empty == 1 && full == 0 && almostfull == 0 && almostempty == 0)
If a write is requested while the FIFO is not full, wr_ack is asserted in the next cycle	@(posedge clk) disable iff (!rst_n) (wr_en && !full) |=> wr_ack
If a write is requested while the FIFO is full (and no read happens), overflow is asserted in the next cycle	@(posedge clk) disable iff (!rst_n) (wr_en && full && !rd_en) |=> overflow
If a read is requested while the FIFO is empty (and no write happens), underflow is asserted in the next cycle	@(posedge clk) disable iff (!rst_n) (rd_en && empty && !wr_en) |=> underflow
data_out stays stable in the next cycle when no valid read happens (rd_en low or FIFO empty)	@(posedge clk) disable iff (!rst_n) !(rd_en && !empty) |=> $stable(data_out)
Whenever count is 0, the empty flag is asserted	@(posedge clk) disable iff (!rst_n) (count == 0) |-> empty
Whenever count equals FIFO_DEPTH, the full flag is asserted	@(posedge clk) disable iff (!rst_n) (count == FIFO_DEPTH) |-> full
Whenever count equals FIFO_DEPTH-1, the almostfull flag is asserted	@(posedge clk) disable iff (!rst_n) (count == FIFO_DEPTH-1) |-> almostfull
Whenever count equals 1, the almostempty flag is asserted	@(posedge clk) disable iff (!rst_n) (count == 1) |-> almostempty
Write pointer wraps around to 0 after a valid write at the last location (FIFO_DEPTH-1)	@(posedge clk) disable iff (!rst_n) ((wr_ptr == FIFO_DEPTH-1) && wr_en && !full) |=> !wr_ptr
Read pointer wraps around to 0 after a valid read at the last location (FIFO_DEPTH-1)	@(posedge clk) disable iff (!rst_n) ((rd_ptr == FIFO_DEPTH-1) && rd_en && !empty) |=> !rd_ptr
Pointers and count never exceed their legal limits (rd_ptr and wr_ptr < FIFO_DEPTH, count <= FIFO_DEPTH)	@(posedge clk) disable iff (!rst_n) (rd_ptr < FIFO_DEPTH && wr_ptr < FIFO_DEPTH && count <= FIFO_DEPTH)

Feature	Assertion
When reset (rst_n) is asserted, all outputs and pointers return to their reset values (count, wr_ptr, rd_ptr, data_out, wr_ack, overflow, underflow, full, almostfull, almostempty = 0 and empty = 1)	always_comb if (!rst_n) assert final (count == 0 && wr_ptr == 0 && rd_ptr == 0 && data_out == 0 && wr_ack == 0 && overflow == 0 && underflow == 0 && empty == 1 && full == 0 && almostfull == 0 && almostempty == 0)
If a write is requested while the FIFO is not full, wr_ack is asserted in the next cycle	@(posedge clk) disable iff (!rst_n) (wr_en && !full) |=> wr_ack
If a write is requested while the FIFO is full (and no read happens), overflow is asserted in the next cycle	@(posedge clk) disable iff (!rst_n) (wr_en && full && !rd_en) |=> overflow
If a read is requested while the FIFO is empty (and no write happens), underflow is asserted in the next cycle	@(posedge clk) disable iff (!rst_n) (rd_en && empty && !wr_en) |=> underflow
data_out stays stable in the next cycle when no valid read happens (rd_en low or FIFO empty)	@(posedge clk) disable iff (!rst_n) !(rd_en && !empty) |=> $stable(data_out)
Whenever count is 0, the empty flag is asserted	@(posedge clk) disable iff (!rst_n) (count == 0) |-> empty
Whenever count equals FIFO_DEPTH, the full flag is asserted	@(posedge clk) disable iff (!rst_n) (count == FIFO_DEPTH) |-> full
Whenever count equals FIFO_DEPTH-1, the almostfull flag is asserted	@(posedge clk) disable iff (!rst_n) (count == FIFO_DEPTH-1) |-> almostfull
Whenever count equals 1, the almostempty flag is asserted	@(posedge clk) disable iff (!rst_n) (count == 1) |-> almostempty
Write pointer wraps around to 0 after a valid write at the last location (FIFO_DEPTH-1)	@(posedge clk) disable iff (!rst_n) ((wr_ptr == FIFO_DEPTH-1) && wr_en && !full) |=> !wr_ptr
Read pointer wraps around to 0 after a valid read at the last location (FIFO_DEPTH-1)	@(posedge clk) disable iff (!rst_n) ((rd_ptr == FIFO_DEPTH-1) && rd_en && !empty) |=> !rd_ptr
Pointers and count never exceed their legal limits (rd_ptr and wr_ptr < FIFO_DEPTH, count <= FIFO_DEPTH)	@(posedge clk) disable iff (!rst_n) (rd_ptr < FIFO_DEPTH && wr_ptr < FIFO_DEPTH && count <= FIFO_DEPTH)